CMPE311 Project 2

Document name: CMPE311 Project 2: Cyclic Executive Lab Report

Document reference: comeauDulcey.CMPE311.project2.CyclicExecutive

Revision: B. Supersedes Revision A, comeauDulcey.CMPE311.project1.AsynchTasking [1].

Date of publication: September 29, 2026

Change authority: ECO 0.1 [2], documented by ECN-001 [3].

LEAD ENGINEER: Dulcey Comeau, UMBC

STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: An Arduino Uno R3 based embedded system that blinks two LEDs at independently User-configurable intervals set over the serial-monitor, with all work scheduled by a polled, round-robin cyclic executive that dispatches tasks from a function pointer table.

DESCRIPTION: This revision implements Engineering Change Order ECO 0.1 [2] against the PROJECT-ASYNC design [1]. The System's external behavior, hardware, Customer-requirements and HL-requirements are unchanged. The software is restructured so that each asynchronous activity from PROJECT-ASYNC (blinking LED1, blinking LED2, and the serial-monitor dialog) is a separate run-to-completion task. A task table holds, for each task, a pointer to its function, its dispatch period, the time it last ran, and an enable flag. A dispatcher called from loop() visits every table entry in fixed index order on each cycle and calls a task through its function pointer when the task is enabled and its period has elapsed. No real-time operating system or third-party multitasking library is used.

RESULT SUMMARY: The revised design continues to satisfy HL.1-HL.6 and EC.1-EC.5 of [1] and satisfies the new educational requirements ER.1.0-ER.1.2 (Table 2). The regression tests T.0-T.10 of [1] are retained unchanged and five tests, T.11-T.15, are added to evaluate the cyclic executive. Before hardware testing, the software was cross-compiled for the ATmega328P (4,020 bytes of flash, 256 bytes of RAM) and exercised in a host simulation that reproduced the Table 5 dialog of [1] with exact 300/300 ms and 800/800 ms on/off times. Hardware results and evidence are recorded in the Testing Results section.

REVISION HISTORY AND ENGINEERING CHANGES

Table 1. Revision history

Rev	Document	Date	Change
A	comeauDulcey.CMPE311.project1.AsynchTasking	September 2026	Initial release (PROJECT-ASYNC).
B	comeauDulcey.CMPE311.project2.CyclicExecutive	September 29, 2026	Cyclic executive task manager per ECO 0.1 [2] and ECN-001 [3].

SUMMARY OF CHANGES FROM REVISION A

The sections of this document either restate Revision A [1] by reference or describe what changed. Sections marked "Changed" or "New" below contain the only engineering differences.

Section	Status	Change
Title and Date	Changed	New document name, reference, revision and date.
References and Glossary	Changed	References [1]-[5] added; cyclic executive terms and ECR/ECO/ECN acronyms added.
Requirements: Customer, HL, EC	Unchanged	Identical to [1]; referenced, not restated.
Requirements: Educational	New	ER.1.0-ER.1.2 from the PROJECT-CE handout [4] (Table 2).
Design: hardware and schematic	Unchanged	Identical to [1] Figure 1 and Table 6.
Design: software architecture	Changed	Cyclic executive with function pointer task table replaces the direct calls in loop() (Figure 1, Tables 3-4).
Testing and Validation	Changed	T.0-T.10 retained; T.11-T.15 added (Table 5); pre-hardware verification added (Table 6).

REFERENCES AND GLOSSARY

REFERENCES

[1] comeauDulcey.CMPE311.project1.AsynchTasking, "CMPE311 Project 1: Asynchronous Tasking Lab Report," Revision A. The PROJECT-ASYNC System Design Document.

[2] ECO 0.1, 1/1/2030, Requestor: Prof. Kidd. Engineering Change Order to use a round-robin cyclic executive task manager (PROJECT-CE handout [4], Figure 1).

[3] comeauDulcey.CMPE311.ECN-001, "Engineering Change Note ECN-001: Cyclic Executive Task Manager."

[4] CMPE311 PROJECT#2: PROJECT-CE lab handout (projectTaskMgmtCE.pdf).

[5] J. Beningo, "Task Scheduling with Function Pointers," https://www.beningo.com/152-task-scheduling-with-function-pointers/

The remaining references are identical to [1] REFERENCES section.

Project source repository: *[insert your GitHub repository URL here]* (per EC.2 of [1]).

DEFINITIONS

The definitions are identical to [1] DEFINITIONS section, with the following additions:

"PROJECT-ASYNC": Project #1, the previous project upon which this project is based, documented in [1].

"PROJECT-CE": Project #2, this project.

"Cyclic executive": A task manager that repeatedly runs a fixed sequence of tasks in a loop, without preemption.

"Task": A function with no arguments and no return value that runs to completion and returns quickly without waiting.

"Task table": The array taskTable[] of task control blocks; each entry holds a function pointer, period, last-run time and enable flag.

"Dispatcher": The function dispatchTasks() that visits the task table and calls each due task through its function pointer.

"Round-robin": Every task is visited once per dispatcher cycle, always in the same index order, with none given priority.

"Polled": Tasks are started by the dispatcher checking the time, not by interrupts.

ACRONYMS AND ABBREVIATIONS

The acronyms are identical to [1] ACRONYMS AND ABBREVIATIONS section, with the following additions:

CE: Cyclic Executive.

ECR: Engineering Change Request.

ECO: Engineering Change Order.

ECN: Engineering Change Note.

ER: Educational Requirement (PROJECT-CE).

SDD: System Design Document.

TCB: Task Control Block (one entry of the task table).

REQUIREMENTS

CONVENTIONS

The conventions are identical to [1] CONVENTIONS section. In addition, the PROJECT-CE educational requirements are labeled “ER.#[.#]”.

CUSTOMER REQUIREMENTS

The customer requirements, C1 through C6, are identical to that of PROJECT-ASYNC. See [1] REQUIREMENTS section, Part I, Table 1. The high-level requirements derived from them, HL-I.1 through HL-I.6, are likewise identical; see [1] REQUIREMENTS section, Part I.

HIGH-LEVEL TECHNICAL REQUIREMENTS

The high-level technical requirements, HL.1 through HL.6, are identical to that of PROJECT-ASYNC. See [1] REQUIREMENTS section, Part II, Table 3. ECO 0.1 [2] states that there is no change to the Customer-requirements or HL-requirements.

EDUCATIONAL CONSTRAINTS

The educational constraints, EC.1 through EC.5, are identical to that of PROJECT-ASYNC. See [1] REQUIREMENTS section, Part II, Table 4. The example dialog the System must perform (EC.4) is identical; see [1] Table 5.

EDUCATIONAL REQUIREMENTS (NEW)

The following requirements are added by the PROJECT-CE handout [4] to implement ECO 0.1 [2].

Table 2. PROJECT-CE educational requirements

ID	Educational requirement
ER.1.0	Implement a cyclic executive task manager based upon a function pointer table to dispatch the asynchronous tasks created in PROJECT-ASYNC.
ER.1.1	The system testing and validation requirements must contain those defined in PROJECT-ASYNC.
ER.1.1.1	The testing and validation requirements must be extended to contain any additional tests necessary to properly evaluate and demonstrate the function of the modified system.
ER.1.2	The final report must be a new revision of the formal design document produced in PROJECT-ASYNC, noting any changes made to the design.

ER.1.1 and ER.1.1.1 are satisfied by the Testing and Validation section of this document. ER.1.2 is satisfied by this document and its Revision History section.

DESIGN

DESIGN PRE-REQUISITES

The design pre-requisites are identical to [1] DESIGN PRE-REQUISITES section, items 1-3 (ELEGOO Uno R3, two LEDs with 470 Ω resistors on digital pins 2 and 3, and Arduino IDE 2.3.3 or later).

DEVELOPMENT PLATFORM

The development platform is identical to [1] DEVELOPMENT PLATFORM section. The software was compiled with avr-gcc 7.3.0, the compiler version supplied by the Arduino AVR Boards core used by Arduino IDE 2.3.3 (EC.1).

CIRCUIT SCHEMATIC

Unchanged. ECO 0.1 affects software only. The circuit schematic and connections are identical to [1] Figure 1 and Table 6: LED1 on digital pin 2 and LED2 on digital pin 3, each through a 470 Ω resistor to a common ground.

SOFTWARE ARCHITECTURE (CHANGED)

In PROJECT-ASYNC [1], loop() called serviceLed(led1), serviceLed(led2) and handleInput() directly. In PROJECT-CE, loop() calls only the dispatcher. The three activities become tasks registered in a function pointer table, and the dispatcher decides when each one runs (Figure 1).

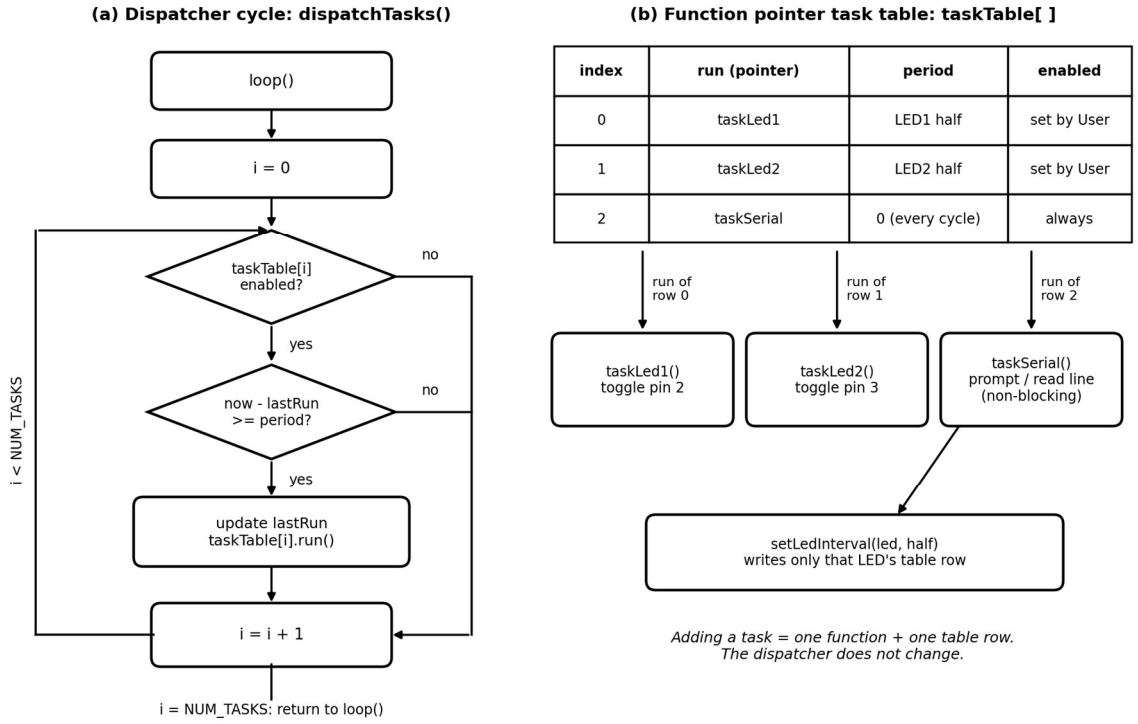


Figure 1. Cyclic executive: (a) dispatcher cycle, (b) function pointer task table.

Task control block and task table

Each task is described by one Task structure. All tasks share the signature void task(void), so a single pointer type, TaskFunction, can hold any of them.

```
typedef void (*TaskFunction)(void);
typedef struct {
  TaskFunction  run;      // pointer to the task function
  unsigned long period;   // msec between dispatches; 0 = every cycle
  unsigned long lastRun;  // millis() of last dispatch
  bool          enabled;  // false = skipped by the dispatcher
} Task;
```

Table 3. Task table (taskTable[])

Index	Function	Period	Enabled	Work performed
0	taskLed1	LED1 interval / 2	After LED1 is set	Toggles digital pin 2.
1	taskLed2	LED2 interval / 2	After LED2 is set	Toggles digital pin 3.
2	taskSerial	0 (every cycle)	Always	Prints the next prompt once, reads any available characters without waiting, and on a complete line validates it and calls setLedInterval().

Dispatcher

dispatchTasks() is a polled, round-robin cyclic executive. On each call it visits taskTable[0] through taskTable[NUM_TASKS - 1] in order. For each enabled task whose period has elapsed (now - lastRun >= period), it advances lastRun and calls the task through its function pointer, taskTable[i].run(). Every task is visited once per cycle and none has priority over another.

Table 4. Software changes from PROJECT-ASYNC

Element	PROJECT-ASYNC (Rev A)	PROJECT-CE (Rev B)
loop()	Calls serviceLed(led1), serviceLed(led2), handleInput() directly.	Calls only dispatchTasks().
Scheduling	Each LED checks its own timer inside serviceLed().	The dispatcher checks every task's period from the task table.
LED state	struct Led {pin, halfPeriod, lastToggle, on} per LED.	Period and timing held in the task table; each LED task only toggles its pin.
Serial dialog	handleInput()	taskSerial(); logic unchanged, now dispatched as a task.
Setting an interval	Writes the Led struct.	setLedInterval() writes only that LED's task table row and enables it.
Timing update	lastToggle = now	Fixed-rate: lastRun += period, with resynchronization after an overrun.
Invalid interval	Rejected values < 1.	Rejects values < 2, so the half-period is never 0 ms.
Instrumentation	None.	Optional CE_INSTRUMENT build flag reports cycle and task times (T.13).

Element	PROJECT-ASYNC (Rev A)	PROJECT-CE (Rev B)
Source file	projectAsync.ino	projectCE.ino (Appendix A)

DESIGN RATIONALE AND ASSUMPTIONS

The rationale and assumptions of [1] DESIGN RATIONALE AND ASSUMPTIONS section still apply (bare-metal millis() timing, interval = full period, current limiting). The following are added for the cyclic executive:

- Function pointer table (ER.1.0): tasks are data, not code in loop(). Adding a future task means writing one void(void) function and adding one row to taskTable[]; dispatchTasks() never changes. This is the labor and cost reduction named as the reason for ECO 0.1 [2].
- Run-to-completion rule: every task must return quickly and must never wait (no delay(), no loops waiting for input). Because the executive is non-preemptive, one blocking task would stall every other task and break HL.4.
- Round-robin, no priority: with three short tasks and a worst-case cycle time far below the smallest LED half-period, every task is serviced within one cycle of becoming due, so priorities are unnecessary.
- Fixed-rate timing (HL.6): lastRun is advanced by exactly one period rather than set to the current time, so small dispatch delays do not accumulate and the long-term blink rate does not drift. If a task falls behind by more than one period, lastRun is resynchronized to the current time so the task does not fire a burst of catch-up toggles.
- Independence (Table 5 of [1]): setLedInterval() writes only the selected LED's table row, so setting one LED cannot change the other LED's period or phase.
- Rollover safety: all time comparisons use unsigned subtraction (now - lastRun), which remains correct when the 32-bit millis() counter wraps after about 49.7 days.
- Serial output bound: each response plus the next prompt is shorter than the 64-byte serial transmit buffer, so Serial.print() returns without waiting at 9600 baud and taskSerial() stays short.
- Resource use: 4,020 bytes of 32,256 bytes of flash (12.3%) and 256 bytes of 2,048 bytes of RAM (12.5%). The task table itself uses 33 bytes of RAM.
- Assumptions: identical to [1] (Newline line ending, positive integer input, 16 MHz clock, correct LED polarity).

Program flow (pseudocode)

```
setup():
    configure pins 2 and 3 as OUTPUT, drive both LOW
    Serial.begin(9600)

loop():
    dispatchTasks()

dispatchTasks():                        // one round-robin cycle
    for i = 0 to NUM_TASKS - 1:
        if taskTable[i].enabled and now - taskTable[i].lastRun >= taskTable[i].period:
            advance taskTable[i].lastRun by one period
            taskTable[i].run()          // call through function pointer

taskLed1() / taskLed2():   toggle the LED pin
taskSerial():              print prompt once; read available chars;
                           on a full line: validate, then setLedInterval(led, value / 2)
```

TESTING AND VALIDATION

DESCRIPTION: The tests below demonstrate that the revised System still satisfies HL.1-HL.6 and EC.1-EC.5 of [1] after the change to a cyclic executive, and that the cyclic executive itself meets ER.1.0: tasks are dispatched from a function pointer table in round-robin order, the dispatcher is fast enough not to disturb the LED timing, new tasks can be added without changing the dispatcher, and invalid input is rejected without affecting either LED.

TESTING PLATFORM

The testing platform is identical to [1] TESTING PLATFORM section, items 1-4 (the Figure 1 testbed of [1], the serial-monitor at 9600 baud with Newline, a timer or camera for timing, and a host computer with repository access). In addition:

1. An instrumented build of projectCE.ino with CE_INSTRUMENT set to 1, used only for T.13. The delivered build has CE_INSTRUMENT set to 0.

TESTING AND VALIDATION REQUIREMENTS

Regression tests (retained, per ER.1.1)

The testing and validation requirements T.0 through T.10 are identical to that of PROJECT-ASYNC. See [1] TESTING AND VALIDATION REQUIREMENTS section, Table 7. They are re-executed in full on the PROJECT-CE build of Appendix A.

Additional tests (new, per ER.1.1.1)

Table 5. Additional testing and validation requirements

ID	Requirement	Verifies
T.11	Every task of The System must be dispatched through a function pointer stored in the task table, and loop() must call only the dispatcher. Verified by source inspection.	ER.1.0
T.12	The dispatcher must visit every task table entry once per cycle in fixed index order, with no task skipped while enabled and due, and every task must run to completion without waiting (no delay() or wait loops). Verified by source inspection.	ER.1.0, HL.4
T.13	With the instrumented build, while the Table 5 dialog of [1] is performed and both LEDs blink, the reported worst-case dispatcher cycle time must be no greater than 5 ms (under 2% of the smallest tested half-period of 300 ms).	ER.1.0, HL.4, HL.6
T.14	A new periodic task must be addable by writing one task function and adding one task table entry, with no change to dispatchTasks(), and without changing the LED timing of T.4.	ER.1.0, ECO 0.1 reason
T.15	Entering an invalid LED number (not 1 or 2) or an interval below 2 ms must produce an error message and return to the "What LED?" prompt, and must not change the rate of either LED that is already blinking.	HL.3, HL.6

PRE-HARDWARE VERIFICATION

Before hardware testing, the software was cross-compiled for the ATmega328P with avr-gcc 7.3.0 and the Arduino AVR core (no errors or warnings), and the unmodified source was run in a host simulation in which millis(), digitalWrite() and Serial were replaced by test doubles using 32-bit time, as on the AVR. Table 6 lists the results. These results support, but do not replace, the hardware results in Table 7.

Table 6. Pre-hardware verification results

Check	Result
Cross-compile for ATmega328P (EC.1)	Pass: 4,020 bytes flash, 256 bytes RAM; instrumented build also compiles.
Table 5 dialog of [1] (EC.4)	Pass: prompts, echoes and confirmations printed in the required order.
LED2 at 600 ms (T.4)	Pass: every half-period exactly 300 ms; 200 toggles in 60 s (expected 200).
LED1 at 1600 ms (T.4)	Pass: every half-period exactly 800 ms.
LED2 while awaiting input (T.7)	Pass: continued at 300 ms throughout the prompt wait.
LED2 after LED1 was set (T.6)	Pass: LED2 half-period unchanged at 300 ms.
Invalid input: LED 7, interval 0 (T.15)	Pass: error message printed, returned to "What LED?"; LED rates unchanged.
Third task added (T.14)	Pass: 1 s heartbeat task added with 3 lines and no dispatcher change; ran 65 times in 65 s; LED timing unchanged.
millis() rollover at 2^32 ms	Pass: 100 ms task kept exact 100 ms spacing across the wrap; no burst after a large time jump.

TESTING RESULTS

Hardware evidence was recorded on the testbed of [1] Figure 1 running the PROJECT-CE build. The testbed wiring is unchanged; see [1] Figure 2. Videos are hosted online and linked here, since they cannot be embedded in the document.

Video 1: *LINK* Serial-monitor dialog: the Table 5 sequence (LED 2 / 600, then LED 1 / 1600), followed by the invalid entries LED 7 and interval 0.

Video 2: *LINK* Blink timing with an on-screen timer: 10 full cycles per LED, and the rate held constant over about 60 s.

Video 3: *LINK* Both LEDs blinking independently while a new command is typed at the prompt.

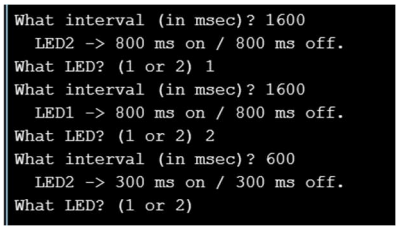


Figure 2. Instrumented build output for T.13 ([CE] cycles, maxCycle_us, per-task times).

Table 7. Results of tests

Test	Result
T.0	Satisfied: all tests performed on the [1] Figure 1 testbed with Arduino IDE 2.3.3 or later.
T.1	Satisfied: two LEDs on digital pins 2 and 3 through 470 Ω resistors; wiring unchanged from [1] Figure 2.
T.2	Satisfied: the PROJECT-CE sketch uploads and runs on the ELEGOO Uno R3 with no modification to the board.
T.3	Satisfied: the System prompts for the LED number and interval and accepts both over the serial-monitor (Video 1).
T.4	Satisfied: LED2 at 600 ms measured *[__]* s over 10 cycles = *[__]* s/cycle; LED1 at 1600 ms measured *[__]* s over 10 cycles = *[__]* s/cycle; on-time = off-time = half the interval, within ±5% (Video 2).
T.5	Satisfied: each LED's rate stayed constant over at least 60 s with no visible drift (Video 2).
T.6	Satisfied: setting LED1 while LED2 was blinking left LED2's rate and phase unchanged (Video 3).
T.7	Satisfied: both LEDs kept blinking at their set intervals while input was typed at the prompt (Video 3).
T.8	Satisfied: the full Table 5 dialog of [1] was reproduced end to end (Video 1).
T.9	Satisfied: the source at *[insert your GitHub repository URL here]* downloads and compiles with the gcc toolchain of Arduino IDE 2.3.3 or later for the ATmega328P and runs on the Reference-platform.
T.10	Satisfied: the software uses only millis(), digitalWrite() and Serial from the Arduino core; no RTOS or third-party multitasking library is present. The cyclic executive is written by The Developer (Appendix A).
T.11	Satisfied: by inspection of Appendix A, taskLed1, taskLed2 and taskSerial are called only through taskTable[i].run() in dispatchTasks(), and loop() contains only the call dispatchTasks().
T.12	Satisfied: by inspection of Appendix A, dispatchTasks() iterates i = 0 to NUM_TASKS - 1 in order on every call; no task calls delay() or waits for input.

Test	Result
T.13	Satisfied: worst-case dispatcher cycle time reported as *[__]* µs after *[__]* cycles, which is no greater than 5 ms (Figure 2).
T.14	Satisfied: a 1 s heartbeat task was added with one function, one table entry and one task ID, with no change to dispatchTasks(); the LED half-periods remained exactly 300 ms and 800 ms (Table 6).
T.15	Satisfied: entering LED 7 printed "Please enter 1 or 2." and entering interval 0 printed "Interval must be 2 msec or more."; both returned to "What LED?" and neither LED's rate changed (Video 1).